Número i nom del curs	25/FOAP/781/0195027/001 IFCD0112 Programació amb llenguatges orientats a objectes i bases de dades relacionals
Mòdul Formatiu que s'avalua	MF0227_3: Programació orientada a objectes / Programación orientada a objetos
UF2404 — Principis de la programació orientada a objectes / Principios de la programación orientada a objetos **UF2405 — Model de programació web i bases de dades / Modelo de programación web y bases de datos** **UF2406 — El cicle de vida del desenvolupament d'aplicacions / El ciclo de vida del desarrollo de aplicaciones**	

PROVA PRACTICA D'AVALUACIÓ DELS APRENENTATGES

Nom i Cognoms de l'alumne/a	
NIF de l'alumne/a	
Data de la prova	
Signatura	

Tipus de prova / Tipo de prueba: Desenvolupament d'una aplicació web amb Django / Desarrollo de una aplicación web con Django
Durada recomanada / Duración recomendada: 5 hores / 5 horas
Puntuació màxima / Puntuación máxima: 10 punts / 10 puntos

Nom formador/a OSCAR BURGOS	**Signatura formador/a**
Signatura i segell del centre de formació	

Aquesta actuació està impulsada i subvencionada pel Servei Públic d'Ocupació de Catalunya (SOC) amb fons rebuts del Ministeri d'Educació, Formació Professional i Esports i del Servei Públic d'Ocupació Estatal (SEPE).

Mòdul: MF0227_3 – Programació orientada a objectes

UF Incloses: UF2404 (POO), UF2405 (Web/Django), UF2406 (Cicle de vida)

🇪🇸 1. INFORMACIÓN GENERAL Y OBJETIVOS

1.1. Contexto del proyecto

Se requiere desarrollar el núcleo funcional y la estructura inicial de un **Sistema de Gestión de Incidencias Técnicas (HelpDesk)** para una empresa simulada. El desarrollo debe aplicar buenas prácticas de Programación Orientada a Objetos (UF2404), un entorno web robusto utilizando Django y ORM (UF2405), así como el control de versiones mediante Git y documentación técnica básica alineada con el ciclo de vida del software (UF2406).

1.2. Requisitos de entrega

- **Repositorio Git:** Debes inicializar un repositorio local, realizar commits atómicos con mensajes descriptivos (Criterio UF2406) y estructurar las ramas (main y desarrollo).
- **Entorno:** Python con entorno virtual (venv) activado y dependencias gestionadas mediante requirements.txt.
- **Formato de entrega:** Al finalizar las 3 horas, comprimirás el proyecto en un archivo .zip (excluyendo la carpeta venv y __pycache__) y entregarás el enlace al repositorio.

🇨🇦 1. INFORMACIÓ GENERAL I OBJECTIUS

1.1. Context del projecte

Es requereix desenvolupar el nucli funcional i l'estructura inicial d'un **Sistema de Gestió d'Incidències Tècniques (HelpDesk)** per a una empresa simulada. El desenvolupament ha d'aplicar bones pràctiques de Programació Orientada a Objectes (UF2404), un entorn web robust utilitzant Django i ORM (UF2405), així com el control de versions mitjançant Git i documentació tècnica bàsica alineada amb el cicle de vida del programari (UF2406).

1.2. Requisits de lliurament

- **Repositori Git:** Heu d'inicialitzar un repositori local, fer *commits* atòmics amb missatges descriptius (Criteri UF2406) i estructurar les branques (main i desarrollo).
- **Entorn:** Python amb entorn virtual (venv) activat i dependències gestionades mitjançant requirements.txt.
- **Format de lliurament:** En finalitzar les 3 hores, comprimireu el projecte en un fitxer .zip (excloent la carpeta venv i __pycache__) i lliurareu l'enllaç al repositori.

Aquesta actuació està impulsada i subvencionada pel Servei Públic d'Ocupació de Catalunya (SOC) amb fons rebuts del Ministeri d'Educació, Formació Professional i Esports i del Servei Públic d'Ocupació Estatal (SEPE).

🇪🇸 2. BLOQUES DE EVALUACIÓN Y TAREAS

Bloque 1: UF2404 - Principios de POO en Python

Antes de saltar a Django, diseña un módulo independiente llamado core_modelos.py que aplique los pilares de la Programación Orientada a Objetos.

1. **Clase Base (TicketTecnico):**
   - Atributos protegidos/privados: _id, _titulo, _estado (Abierto, En Proceso, Cerrado), _prioridad.
   - Métodos para cambiar el estado validando transiciones lógicas (ej: no se puede pasar de Cerrado a En Proceso directamente).
2. **Herencia y Polimorfismo:**
   - Crea dos clases hijas: TicketHardware (con atributo específico componente) y TicketSoftware (con atributo específico sistema_operativo).
   - Sobrescribe un método calcular_sla() de forma que devuelva las horas máximas de resolución según la prioridad y el tipo.
3. **Pruebas unitarias básicas:** Incluye un bloque if __name__ == "__main__": con instancias que demuestren el funcionamiento de las clases y el manejo de excepciones personalizadas (ej. EstadoInvalidoError).

🇨🇦 2. BLOCS D'AVALUACIÓ I TASQUES

Bloc 1: UF2404 - Principis de POO en Python

Abans de saltar a Django, dissenya un mòdul independent anomenat core_modelos.py que apliqui els pilars de la Programació Orientada a Objectes.

1. **Classe Base (TicketTecnico):**
   - Atributs protegits/privats: _id, _titulo, _estado (Obert, En Procés, Tancat), _prioridad.
   - Mètodes per canviar l'estat validant transicions lògiques (ex: no es pot passar de Tancat a En Procés directament).
2. **Herència i Polimorfisme:**
   - Crea dues classes filles: TicketHardware (amb atribut específic componente) i TicketSoftware (amb atribut específic sistema_operativo).
   - Sobreescriu un mètode calcular_sla() de manera que retorni les hores màximes de resolució segons la prioritat i el tipus.
3. **Proves unitàries bàsiques:** Inclou un bloc if __name__ == "__main__": amb instàncies que demostrin el funcionament de les classes i la gestió d'excepcions personalitzades (ex. EstatInvalidError).

Aquesta actuació està impulsada i subvencionada pel Servei Públic d'Ocupació de Catalunya (SOC) amb fons rebuts del Ministeri d'Educació, Formació Professional i Esports i del Servei Públic d'Ocupació Estatal (SEPE).

🇪🇸 Bloque 2: UF2405 - Desarrollo Web y Base de Datos con Django

Crea un proyecto Django llamado helpdesk_project y una aplicación llamada incidencias.

1. **Modelos y ORM (models.py):**
   - Implementa el modelo Tecnico (nombre, especialidad, email).
   - Implementa el modelo Incidencia (relacionado con Técnico mediante ForeignKey o OneToOne según prefieras): título, descripción, fecha_creacion, estado y prioridad (usa choices), y el técnico asignado.
   - Realiza y aplica las migraciones (makemigrations / migrate).
2. **Panel de Administración (admin.py):**
   - Registra los modelos en el panel de administración de Django configurando list_display, filtros por estado y buscador (search_fields) por título o técnico.
3. **Vistas, Formularios y Plantillas (views.py, forms.py, Templates):**
   - Crea una vista basada en funciones o clases (ListView) para listar todas las incidencias.
   - Crea una vista con formulario (CreateView o Formulario de Django) para registrar una nueva incidencia desde la interfaz web.
   - Diseña al menos dos plantillas HTML básicas utilizando herencia de plantillas (base.html) y Bootstrap (o CSS propio) para la lista y el formulario.

🇨🇦 2. (Continuació) / Bloc 2: UF2405 - Desenvolupament Web i Base de Dades amb Django

Crea un projecte Django anomenat helpdesk_project i una aplicació anomenada incidencias.

1. **Models i ORM (models.py):**
   - Implementa el model Tecnico (nom, especialitat, correu).
   - Implementa el model Incidencia (relacionat amb Tècnic mitjançant ForeignKey o OneToOne segons prefereixis): títol, descripció, data_creacio, estat i prioritat (usa choices), i el tècnic assignat.
   - Realitza i aplica les migracions (makemigrations / migrate).
2. **Panell d'Administració (admin.py):**
   - Registra els models al panell d'administració de Django configurant list_display, filtres per estat i cercador (search_fields) per títol o tècnic.
3. **Vistes, Formularis i Plantilles (views.py, forms.py, Templates):**
   - Crea una vista basada en funcions o classes (ListView) per llistar totes les incidències.
   - Crea una vista amb formulari (CreateView o Formulari de Django) per registrar una nova incidència des de la interfície web.
   - Dissenya almenys dues plantilles HTML bàsiques utilitzant herència de plantilles (base.html) i Bootstrap (o CSS propi) per a la llista i el formulari.

Aquesta actuació està impulsada i subvencionada pel Servei Públic d'Ocupació de Catalunya (SOC) amb fons rebuts del Ministeri d'Educació, Formació Professional i Esports i del Servei Públic d'Ocupació Estatal (SEPE).

🇪🇸 Bloque 3: UF2406 - Ciclo de vida, Control de Versiones y Documentación

En esta fase se evalúan los aspectos metodológicos y de calidad del software.

1. **Control de Versiones (Git):**
    - El historial de Git debe reflejar un flujo de trabajo estructurado. Se valorará: Commit inicial del Bloque 1, commits intermedios de modelos en Django, commits de vistas/plantillas, y un commit final de empaquetado.
2. **Documentación y Fichero README.md:**
    - Incluye un archivo README.md en la raíz del proyecto que contenga:
        - Instrucciones de instalación y despliegue local (pasos para crear el entorno virtual, instalar dependencias de requirements.txt, ejecutar migraciones y levantar el servidor).
        - Breve descripción de la arquitectura del proyecto (cómo se relacionan las UF).
3. **Plan de Pruebas / Validación:**
    - Añade un pequeño documento de texto (pruebas.txt) o sección en el README donde indiques 3 casos de prueba funcionales ejecutados (ej: Validación de campos vacíos en el formulario web, transición correcta de estados en POO).

🇨🇦 3. BLOC D'AVALUACIÓ 3: UF2406 - Cicle de vida, Control de Versions i Documentació

En aquesta fase s'avaluen els aspectes metodològics i de qualitat del programari.

1. **Control de Versions (Git):**
    - L'historial de Git ha de reflectir un flux de treball estructurat. Es valorarà: Commit inicial del Bloc 1, commits intermedis de models a Django, commits de vistes/plantilles, i un commit final d'empaquetat.
2. **Documentació i Fitxer README.md:**
    - Inclou un fitxer README.md a l'arrel del projecte que contingui:
        - Instruccions d'instal·lació i desplegament local (passos per crear l'entorn virtual, instal·lar dependències de requirements.txt, executar migracions i aixecar el servidor).
        - Breu descripció de l'arquitectura del projecte (com es relacionen les UF).
3. **Pla de Proves / Validació:**
    - Afegeix un petit document de text (pruebas.txt) o secció al README on indiquis 3 casos de prova funcionals executats (ex: Validació de camps buits al formulari web, transició correcta d'estats en POO).

Aquesta actuació està impulsada i subvencionada pel Servei Públic d'Ocupació de Catalunya (SOC) amb fons rebuts del Ministeri d'Educació, Formació Professional i Esports i del Servei Públic d'Ocupació Estatal (SEPE).

🇪🇸 3. CRITERIOS DE CALIFICACIÓN

- **UF2404 (POO en Python):** 30% (Correcta aplicación de encapsulamiento, herencia, manejo de excepciones y estructura limpia).
- **UF2405 (Django & Base de Datos):** 40% (Modelos funcionales, migraciones correctas, panel de administración operativo, vistas y formularios conectados con éxito).
- **UF2406 (Ciclo de Vida / Git / Docs):** 30% (Uso correcto de Git con commits significativos, presencia de requirements.txt y un README.md claro y completo).

🇨🇦 4. CRITERIS DE QUALIFICACIÓ

- **UF2404 (POO en Python):** 30% (Correcta aplicació d'encapsulament, herència, gestió d'excepcions i estructura neta).
- **UF2405 (Django & Base de Dades):** 40% (Models funcionals, migracions correctes, panell d'administració operatiu, vistes i formularis connectats amb èxit).
- **UF2406 (Cicle de Vida / Git / Docs):** 30% (Ús correcte de Git amb commits significatius, presència de requirements.txt i un README.md clar i complet)

Aquesta actuació està impulsada i subvencionada pel Servei Públic d'Ocupació de Catalunya (SOC) amb fons rebuts del Ministeri d'Educació, Formació Professional i Esports i del Servei Públic d'Ocupació Estatal (SEPE).